Hold-On: Technical Architecture and Build Plan

Oct 30, 2026 · @Nikhil Sharma

1. Purpose and key decisions

Hold-On runs as one Python backend that owns every decision, while Gnani's models listen and speak and Sarvam-30B proposes. This plan names each layer, the Gnani endpoints it calls, and where the vector index fits.

The vector index is a retrieval layer, not the ledger. Promises, deadlines, consent and job locks need transactions and exact lookups, so they live in ordinary Postgres tables. Embeddings sit beside them in pgvector and are used where meaning-based lookup helps (section 4).

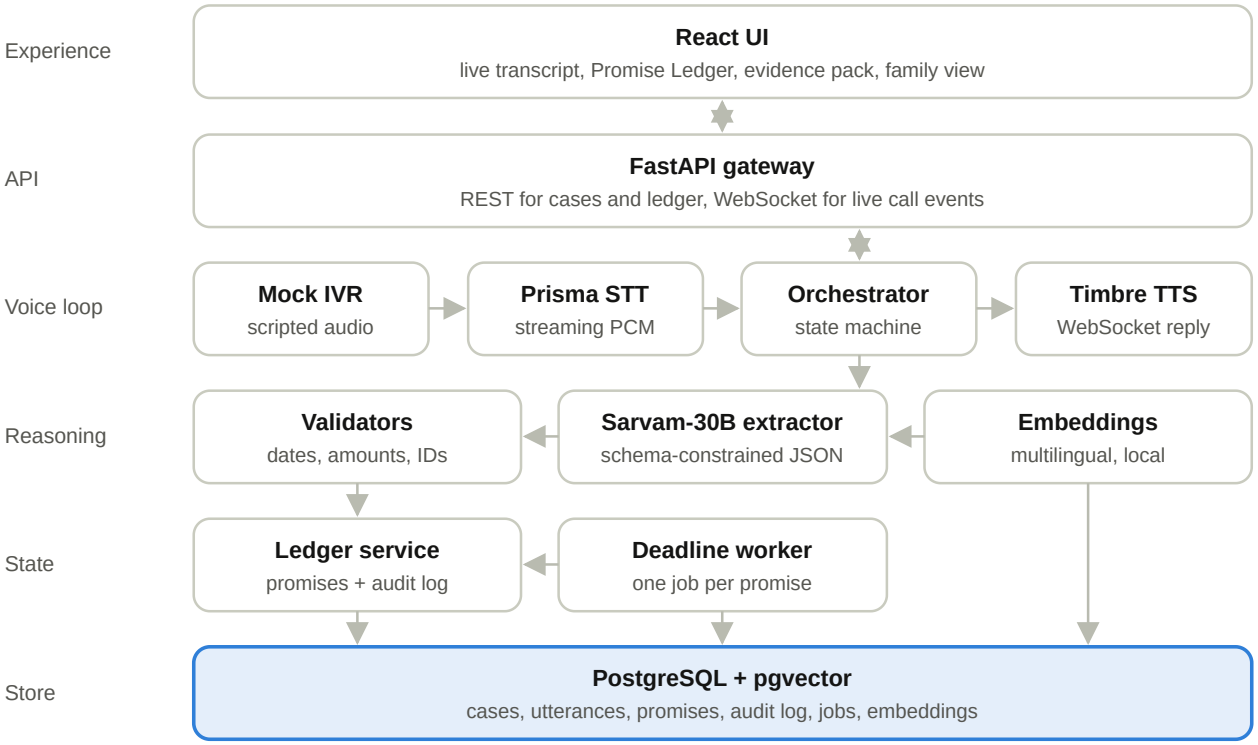
At demo scale, with hundreds of rows, the database is never the latency bottleneck; the speech and model calls are. We use vectors for grounding and correctness, and we do not claim they make the system faster.

2. System architecture

Every decision sits in the backend; Gnani's models only listen and speak, and Sarvam-30B only proposes.

Layer	Choice	Why
Frontend	React + Vite + TypeScript, Tailwind	Quick to build; live panels for transcript, ledger and timeline
Backend	Python 3.11+, FastAPI (async)	Gnani's official Python SDK, ffmpeg tooling and the eval scripts share one language
Realtime	WebSockets: browser to backend, backend to Gnani	Streaming speech-to-text and text-to-speech are WebSocket APIs
Database	PostgreSQL with the pgvector extension	One database for the transactional ledger and the vector index
Scheduler	A `jobs` table claimed with `FOR UPDATE SKIP LOCKED`	Duplicate-safe callbacks with no Redis
Extraction	Sarvam-30B (`sarvamai/sarvam-30b`, Apache 2.0), self-hosted on a cloud VM we rent (any provider) behind an OpenAI-compatible endpoint, plus a fallback model	Schema-constrained JSON; validators decide what is kept
Embeddings	A multilingual sentence-embedding model run locally (for example multilingual-e5-small, 384 dimensions)	Free, handles Hindi and English; verify on your own test pairs
Audio tooling	ffmpeg and numpy	Telephone-quality (8 kHz) conversion and noise mixing for tests

Gnani models listen and speak, Sarvam-30B proposes; the backend decides and records



Highlighted: the single source of truth. Arrows show which part calls or feeds which.

architecture · 6 layers, 12 parts

Read top to bottom: the React app talks only to FastAPI, which drives the orchestrator. Prisma listens, Timbre speaks and Sarvam-30B proposes, and every proposal passes the validators and the user's confirmation before it reaches the ledger in Postgres. The browser never holds a Gnani key, and the orchestrator's replies go back into the mock call through Timbre.

3. Gnani APIs and how we call them

These details come from Gnani's own docs pages and the Sarvam-30B model pages and docs, read on Oct 30, 2026; confirm them again on day 1.

Service	Endpoint	Key settings	Role in Hold-On
Prisma STT, streaming	`wss://api.vachana.ai/stt/v3/stream`	Headers `x-apikey-id`, `language_code` (for example `hi-IN`; Prisma v3.0 also takes a list like `hi-IN,en-IN`), `model` (`prisma-v3.0`, which we use; the default is `prisma-v2.5`), `x-sample-rate` (8000 or 16000), `format` (`verbatim`, `transcribe`, `codemix` on v3.0); audio is raw 16-bit mono PCM in 1,024-byte frames	Far-end audio from the mock IVR and agent; each transcript carries a `latency` value we log
Prisma STT, REST	`POST /stt/v3`	File upload for short clips; the docs give 30 s on one page and 60 s on another, so confirm	Faster-than-real-time evaluation runs

Details that shape the build

- **Keys stay on the server.** A browser WebSocket cannot set the custom headers the STT socket needs, so the backend proxies the connection, which also keeps the key off the client.

Service	Endpoint	Key settings	Role in Hold-On
Timbre TTS	`wss://api.vachana.ai/api/v1/tts`; REST at `/api/v1/tts/inference`, SSE at `/api/v1/tts/sse`	Header `X-APIKey-ID`; JSON fields `text`, `voice`, `model` (`timbre-v3.0`, which we use; `timbre-v2.5` also exists), `language` (`hi-IN`, `hi-en` for Hinglish, `auto`), `speed` (0.85 to 1.15), `audio_config` (8,000 to 48,000 Hz; containers `wav`, `raw`, `mp3`, `ogg`, `mulaw`, `alaw`); the server returns base64 audio chunks, then `complete`	Spoken replies and the callback line; REST or SSE to pre-generate fixed lines

Sarvam-30B	Our own OpenAI-compatible server on a rented cloud VM (any provider), serving `sarvamai/sarvam-30b` with vLLM or SGLang, or a GGUF build with llama.cpp's `llama-server`	Open weights, Apache 2.0, no access request (not gated); Mixture-of-Experts, 30B total and 2.4B active parameters per Sarvam's docs; 64K-token context per Sarvam's API docs (confirm on the model card); custom code, so `trust_remote_code=True`; BF16 needs an 80GB-class GPU (roughly 64 GB of weights, estimated), while the GGUF builds are about 19 GB (Q4_K_M), 26 GB (Q6_K) or 34 GB (Q8_0)	Extraction proposals as JSON; validators decide

- **Audio streams at real-time pace.** The STT socket expects one 1,024-byte frame every 32 ms at 16 kHz (64 ms at 8 kHz), and bursting hurts its voice detection. A 2-minute scripted call therefore takes 2 minutes to stream, so evaluation uses REST for short clips or several streams in parallel (concurrent sessions are capped per organisation).
- **Holds need a reconnect step.** The idle timeout defaults to 60 s (`x-idle-timeout-sec`, 5 to 900) and a session is capped at 15 minutes.
- **Tune voice detection for phone audio.** Gnani's telephony preset is 8 kHz, threshold 0.7, minimum speech 200 ms and minimum silence 500 ms; shorter silence (300 ms) suits IVR prompts. Raising the threshold helps reject hold music.
- **We use Prisma v3.0, so its extras are available.** It takes a list of language codes (`hi-IN,en-IN`) and the `codemix` format for Hinglish, and its `keyterms`, `topic` and `previous_text` hints can carry operator names, ticket words and the previous turn; test whether each one helps before relying on it.
- **Say numbers as words to Timbre.** Gnani warns that raw numbers, IDs, dates and currency are mispronounced, so a `speak_text()` formatter turns ticket #4821 into spoken digits, following Gnani's input formatting guide.
- **Sarvam-30B runs on a VM we rent, for running and testing.** We self-host the open weights on any cloud VM that offers GPUs, and we do not use Sarvam's hosted API, because its docs now list `sarvam-30b` as deprecated there. The weights are public on Hugging Face under Apache 2.0 with no access request, so there is no approval wait. Two ways to serve it: vLLM or SGLang on an 80GB-class GPU following the model card's setup (it includes an extra step before vLLM runs as usual), or a GGUF build with llama.cpp's `llama-server` on a smaller card (for example Q8_0, about 34 GB, on a 48GB GPU; the official repo is `sarvamai/sarvam-30b-gguf`, and some builds need a llama.cpp with `sarvam_moe` support, so test a stock build first). The VM is started for the day-1 Hinglish test, the evaluation runs and the demo recording, and destroyed when idle to save credits. Day-to-day development uses a small local model on our laptops behind the same URL. Transcripts sent to Sarvam-30B stay on our own VM. We request plain JSON, validate it, and do not depend on tool calls. Fall back to Qwen3.5-35B-A3B (Apache 2.0, same size class) on the same VM, behind the same schema, if Sarvam-30B fails the Hinglish test.
- **Versions are configurable.** We use v3.0 of both Prisma and Timbre; keep model names in environment variables and name the exact versions in the submission. From your console, speech cost about ₹0.46 per minute (STT) and ₹2.75 per 1,000 characters (TTS), so credits are not the constraint.

4. Data layer: Postgres with pgvector

Postgres is the source of truth, and pgvector adds a meaning-based index on the same rows.

Table	Holds
`cases`	Owner, language, goal, approved information, follow-up consent, status
`calls`	Case, scenario id, audio file, start and end, Gnani `x-api-request-id` values
`utterances`	Call, speaker role, start and end (from our own stream position, since Gnani returns each segment's duration), raw and normalised text, segment id, latency, embedding `vector(384)`
`promises`	Case, source utterance, quoted text, type, amount, raw date phrase, `due_at` (Asia/Kolkata), one of five statuses, user confirmation, verification result
`promise_events`	Append-only log of every state change and decision, with the rule that fired
`jobs`	Promise, attempt number, unique `idempotency_key`, run time, claim time, result
`kb_chunks`	Escalation-guide passages with source URL, checked date and embedding

Four jobs for the vector index

1. **Quote grounding.** For each proposed promise, retrieve the nearest utterances; it passes only if the quoted words appear exactly in a retrieved turn.
2. **Case memory for callbacks.** Retrieve the most relevant earlier turns and promises for a case to build the follow-up script, which matters once a case spans several calls.
3. **Escalation guide.** Retrieve the matching passage from the checked official sources and show it with its link and checked date.
4. **IVR option matching.** Embed the user's goal and the heard menu options to rank candidates; a rule check and the state machine still choose the key to press.

Vectors never hold the ledger, deadlines, consent or job locks; those stay in plain SQL.

Embeddings. A multilingual sentence-embedding model runs locally on CPU, with its name stored beside every vector so we can re-embed later; distance is cosine. Check it on about 30 of your own Hindi and Hinglish pairs before relying on it. Setup: Postgres 16 with the pgvector extension in Docker Compose; skip the HNSW index at demo scale and add it only if queries slow down. If you host the database, confirm the provider supports pgvector.

5. Core pipeline, from call to evidence

Only the far end of the call (IVR and agent) goes to Prisma; our side is Timbre speech we generate ourselves, so every speaker role is known from the scenario and nothing relies on diarization.

1. **Intake.** The UI posts a case; language, goal, approved information and follow-up consent are saved.
2. **Call start.** The orchestrator loads a scenario (menu tree, hold segment, agent script) and plays the far-end audio to Prisma as PCM frames at real-time pace.
3. **Utterances.** Each transcript becomes a row with its speaker role, start and end from our stream position, and an embedding.
4. **IVR step.** Heard menu options are ranked against the goal (rules first, vectors as a tie-break) and the chosen key goes into the mock IVR.

5. **Hold.** The scenario flags hold; the orchestrator keeps the socket alive and reconnects before the idle timeout or the 15-minute cap.
6. **Agent turn.** The last few turns go to Sarvam-30B with a strict JSON schema (ticket ID and candidates with quote, speaker, type, amount, raw date phrase); replies are spoken by Timbre.
7. **Validation.** Schema check, quote found in a retrieved turn, amount and date parsed, role checked; "we will look into it" is filed as a statement, never a promise.
8. **Confirmation.** The user confirms in the UI; only then does a `promises` row exist, as pending with a `due_at`.
9. **Deadline worker.** Every few seconds it claims due jobs with `FOR UPDATE SKIP LOCKED`, rechecks status and consent, and creates one callback; a demo-clock endpoint advances simulated time.
10. **Callback and evidence.** A second mock call opens with a Timbre line citing the ticket and promise; the result goes to `promise_events`, and the evidence pack is built from the database and exported as a PDF.

Extraction settings. Use the sampling settings on Sarvam-30B's model card (confirm them on day 1; start with a low temperature such as 0.2 for extraction), with thinking turned off for short extractions (the chat template has an `enable_thinking` switch), and retry up to twice on invalid JSON.

Dates. A small parser handles Hindi and English number words ("paanch", "teen") and distinguishes working days (Monday to Friday minus a configured holiday list) from calendar days, always in Asia/Kolkata. The raw phrase is stored beside the normalised date, and an unclear start point sends the user an absolute date to confirm.

6. Frontend and API contracts

The React app has six screens: intake; live call (transcript with speaker chips and a hold indicator); Promise Ledger (quote, speaker, timestamp, due date, status chip, confirm button); case timeline with the evidence pack; a read-only family view; and demo controls (scenario picker, noise toggle, simulated clock).

Endpoint	Purpose
`POST /cases`, `GET /cases/{id}`	Create a case with consent; read its state
`POST /calls`	Start a scenario for a case
`WS /ws/calls/{id}`	Live events: `speech_start`, `processing`, `final`, `menu_pressed`, `hold_started`, `promise_proposed`, `call_ended`
`POST /promises/{id}/confirm`	User confirms or corrects a proposed promise
`POST /demo/clock`	Advance simulated time and run the deadline check
`GET /cases/{id}/evidence.pdf`	The evidence pack
`GET /family/{token}`	Read-only status for a trusted family member

Prisma sends whole transcripts once a speech segment ends, so the live view shows a "listening" state on `speech_start` and `processing`, then the final text.

7. Testing and evaluation harness

An `eval/` folder holds labelled JSONL (transcript, speaker, ticket ID, amount, date, promise type, expected next state) and a runner that calls the same pipeline functions headlessly, writing per-field results to CSV with clean and noisy audio split.

- **Audio matrix.** Build variants with ffmpeg: 8 kHz telephone conversion with a μ-law codec, noise mixed at fixed signal-to-noise levels, and a low-pass filter for a speakerphone feel. Add hold music, long silences, keypad tones, interruptions and mixed Hindi and English. Use real recordings from consenting people for most clips and Timbre-generated audio for a minority, because Timbre talking into Prisma is a friendly loop.
- **Run time.** Streaming runs at real-time pace, so use REST for short clips and a few parallel streams, and run the full streaming suite only a handful of times.
- **Unit tests.** Date and amount parsers, validators, state transitions, consent rechecks, and idempotency (two workers racing for one job must claim it once).
- **Latency log.** Record a timestamp at each stage: frame sent, `speech_start`, `processing`, transcript (with Gnani's `latency`), Sarvam reply, validator done, first Timbre chunk. Report median and p95.
- **Regression gate.** The automatic callback stays disabled in the demo build until the deterministic suite passes.

Targets are set after the first run, and every reported figure carries its sample count.

8. Deployment, security and the vibe-coding workflow

Deployment. Run web, API and Postgres with pgvector locally in Docker Compose for the demo recording. The only remote piece is the rented VM that runs Sarvam-30B; the backend calls it through one OpenAI-compatible URL kept in an environment variable. A hosted copy (static frontend, container backend) is optional if a free tier fits, and a recorded working run is the backup if a live take fails.

Security

- API keys live in server-side environment variables only, never in React and never in logs.
- Transcripts are untrusted input: what an agent says cannot change the schema, call a tool or trigger a callback; only validators and the user's confirmation can.
- `speak_text()` refuses to voice OTP, PIN, password or card-number patterns.
- Data is synthetic, voices are consented, the telecom is fictional, and consent is rechecked before every callback.

Build order for the coding assistant

1. Repo skeleton, Docker Compose and migrations for the section 4 tables.
2. Gnani adapters (STT stream, STT REST, TTS) with a smoke-test script that prints the transcript, latency and credit change.
3. Mock IVR engine driven by scenario JSON (menu tree, hold segment, agent script).
4. Utterance pipeline: transcripts to rows to embeddings.
5. Extraction and validators, with the tests written first.
6. Ledger API and UI with confirmation.

7. Deadline worker, callback and demo clock.
8. Evaluation runs and the evidence pack, then freeze.

Target steps 1 to 3 by Nov 1, 4 to 6 by Nov 4, 7 by Nov 6, and step 8 with the feature freeze by Nov 7; the demo is recorded on Nov 8 and the entry submitted on Nov 9.

Rules for every prompt. Paste the schemas and contracts from sections 4 and 6; ask for one module and its tests at a time and run them before moving on; never call Gnani from the frontend; keep model names in environment variables; commit after each green step. Review generated code for any path that writes to `promises` without a user confirmation.

9. Open decisions and risks

Six choices need an answer in the first days, because later steps depend on them.

Decision or risk	Options	Decide by
Sarvam-30B VM	Pick any cloud VM provider and confirm a new account can start a GPU instance right away (some need a quota request) and when the credit expires. Choose an 80GB-class GPU with vLLM, or a 48GB-class GPU with a GGUF build. Destroy the VM when idle. Fallback: Qwen3.5-35B-A3B on the same VM behind the same JSON schema	Oct 31
Prisma and Timbre versions	Decided: v3.0 for both. Confirm with Gnani that v3.0 is acceptable for the challenge and name the exact versions in the submission	Oct 31
Audio path for the demo	16 kHz clean path, 8 kHz telephone path, or both; test both before choosing	Nov 1
Concurrent stream cap	Ask on Discord; if it is low, run streaming evaluation sequentially	Nov 1
Embedding quality on Hinglish	Check about 30 pairs; if weak, ground quotes by exact text match only	Nov 3
Database hosting	Stay local in Docker, or confirm a host supports pgvector	Nov 3

Sources

Gnani pages opened on Oct 30, 2026; the measured speech costs come from the team's own console screenshots. The Sarvam-30B details come from its Hugging Face pages, Sarvam's API docs and launch coverage, read on Oct 30, 2026.

- Gnani Prisma realtime streaming docs
- Gnani Timbre realtime TTS docs
- sarvamai/sarvam-30b model page (Hugging Face)
- sarvamai/sarvam-30b-gguf (Hugging Face)
- Sarvam API docs: Sarvam-30B (hosted API deprecated)